The midterm, question by question

TC1037 · Algorithmic Thinking · Session 11 · debrief

Dr. Leopoldo Rodríguez Salazar

Tuesday 3 November 2026

Tecnológico de Monterrey · Campus Querétaro · Group 510

Open Canvas now.

Every graded item has a written comment from me.

- Quick exams 1–4, homework 1–5, in-class 1–3 — all of them have feedback, not just a number.
- Read **two** of them: your worst score, and your best. The comment says exactly which line cost you the points.
- Write down **one** thing you will do differently. That is the whole exercise.

Five minutes. Then we start with the midterm.
Questions about a specific grade: book an asesoría, link in the Canvas announcement.

Today

First hour — your exam

- Part I: all six questions, both versions, with the trace
- Part II: the reference program, piece by piece
- where the points actually went

Everything on this screen is also on the site: **exams page** — both papers, both solutions, and a Python key you can run in Thonny.

Then

- what to do with this result
- week 7: **lists**

How to use this hour

Have your paper in front of you. For every question: look at *your* answer first, then at the trace. The question to answer is not "what is the right answer" — it is *where did my reading go wrong*.

How Part I and Part II were scored

Part I — 6 questions × 5 pts

All or nothing. Every line of the output, in the right order. A missing line, an extra line or a wrong order is 0 for that question — because a program that prints *almost* the right thing is wrong.

Part II — 70 pts, the academia's rubric

function with `while`	23
function with `for`	23
`menu()`	3
`main()`: loop, validation, calls, labels	20
last line calls `main()`	1
technical requirements	−2 each

Each piece on the six-level scale: Excelente 100 % · Muy bien 85 · Regular 70 · Deficiente 50 · Muy deficiente 20 · Sin evidencia 0.

```
print(7 + 8 // 3 * 2 - 9 % 4 *
    * 2)
```

Answer: 2

One operator at a time, by precedence:

1 `**` first: `4 ** 2` → 16

2 then `// * %`, left to right:

`8 // 3` → 2

`2 * 2` → 4

`9 % 16` → 9

3 last `+ -`: `7 + 4 - 9` → 2

The trap: `9 % 4 ** 2` is `9 % 16`, not `(9 % 4) ** 2`. Power beats remainder.

```
print(9 + 7 // 2 * 3 - 10 % 3
    ** 2)
```

Answer: 17

Same ladder:

1 `3 ** 2` → 9

2 `7 // 2` → 3

`3 * 3` → 9

`10 % 9` → 1

3 `9 + 9 - 1` → 17

The trap: `10 % 3 ** 2` is `10 % 9`, not `(10 % 3) ** 2`. And `7 // 2` is 3, not 3.5 — `//` throws the decimals away.

```
1 print(not 3 > 2 or 4 == 4.0
      and 5 < 5)
```

Answer: `False`

Order: comparisons, then `not`, then `and`, then `or`.

1. `3 > 2` → `True`, `4 == 4.0` → `True`, `5 < 5` → `False`
2. `not True` → `False`
3. `True and False` → `False`
4. `False or False` → `False`

Two traps: `not` binds only to `3 > 2`, not to the whole line; and `4 == 4.0` is `True` — an int and a float with the same value are equal.

Part I · Question 2 · boolean operators [B]

```
1 print(not 5 < 4 and 3 == 3.0
      or 2 > 2)
```

Answer: `True`

1. `5 < 4` → `False`, `3 == 3.0` → `True`, `2 > 2` → `False`
2. `not False` → `True`
3. `True and True` → `True`
4. `True or False` → `True`

The trap: reading it as `not (5 < 4 and 3 == 3.0)` gives `True` too — by luck. Do it in the right order anyway: on version A the same shortcut gives the wrong answer.

```python
x = 12
y = 7
if x % y > 4:
    print("A", x - y)
elif x > y and y % 2 == 1:
    print("B", x + y)
else:
    print("C")
print("end")
```

Answer:

```
A 5
end
```

`12 % 7` → 5, and `5 > 4` is `True` → the `if` branch runs, prints `A` and `12 - 7`.

Two traps. The `elif` is *never tested* once the `if` is true — its condition is also true here, and it does not matter. And `print("end")` is outside the structure, so it always runs: **two lines**, not one.

```
x = 14
y = 5
if x % y < 3:
    print("A", x + y)
elif x > y and x % 2 == 0:
    print("B", x - y)
else:
    print("C")
print("end")
```

Answer:

```
B 9
end
```

`14 % 5` → 4, and `4 < 3` is `False` → go to the `elif`: `14 > 5 and 14 % 2 == 0` → `True` → prints `B` and `14 - 5`.

Same two traps: the `else` does not run once a branch matched, and `end` always prints.

Part I · Question 4 · functions with a `for` [A]

```
1  def f(a, b):
2      for i in range(b):
3          a = a + i
4      return a
5
6  def main():
7      a = 3
8      b = 4
9      c = f(b, a)
10     print(a, b, c)
11
12 main()
```

Answer: `3 4 7`

The call crosses the arguments: `f(b, a)` with `b`=4 and `a`=3, so *inside* `f`: `a`=4, `b`=3.

`range(3)` → `0, 1, 2`

`i`	`a`
—	4
0	4
1	5
2	7

The `a` and `b` inside `f` are *different variables* from the ones in `main` — `main`'s `a` and `b` never change. Wrong answers seen: `3 4 9` (no crossing), `3 4 10` (`range(3)` read as 0–3), `4 3 7` (printing the inside names).

Part I · Question 4 · functions with a `for` [B]

```
1  def g(a, b):
2      for i in range(1, b):
3          a = a * i
4      return a
5
6  def main():
7      a = 4
8      b = 2
9      c = g(b, a)
10     print(a, b, c)
11
12 main()
```

Answer: `4 2 12`

Crossed again: `g(b, a)` with b=2, a=4 → inside: a=2, b=4.

`range(1, 4)` → 1, 2, 3

i	a
—	2
1	2
2	4
3	12

`range(1, 4)` stops *before* 4. `main` still prints its own a and b: `4 2`.

Part I · Question 5 · the `while` loop [A]

```
1 n = 1
2 total = 0
3 while n < 20:
4     total = total + n
5     n = n * 3
6 print(n, total)
```

Answer: `27 13`

turn	n < 20?	total	n
start	—	0	1
1	yes	1	3
2	yes	4	9
3	yes	13	27
4	**no**	—	stop

The trap: the loop stops *after* `n` has already passed 20. The printed `n` is 27, not 9 — the value that broke the condition is the value you print.

Part I · Question 5 · the `while` loop [B]

```
1 n = 2
2 total = 0
3 while n < 30:
4     total = total + n
5     n = n * 2
6 print(n, total)
```

Answer: `32 30`

turn	n < 30?	total	n
start	—	0	2
1	yes	2	4
2	yes	6	8
3	yes	14	16
4	yes	30	32
5	**no**	—	stop

Same trap: `n` ends at 32, past the limit. `total` adds 2+4+8+16, not 32.

```python
for i in range(10, 0, -3):
    if i % 2 == 0:
        print(i)
    else:
        print(-i)
```

Answer, four lines:

`10` `-7` `4` `-1`

`range(10, 0, -3)` → `10, 7, 4, 1` — down by 3, stopping *before* 0.

i	even?	prints
10	yes	10
7	no	−7
4	yes	4
1	no	−1

The trap: the `else` prints `-i`, which is the *negative* of the value — not a subtraction, and not a skip. Four lines, in this order.

Part I · Question 6 · the `for` loop [B]

```
for i in range(12, 0, -4):
    if i % 3 == 0:
        print(i)
    else:
        print(-i)
```

Answer, three lines:

`12` `-8` `-4`

`range(12, 0, -4)` → `12, 8, 4` — it never reaches 0.

i	% 3 == 0?	prints
12	yes	12
8	no	-8
4	no	-4

Count the lines before you write: three here, four on version A. A fourth line (`0`) is the classic error — `range` stops *before* the end value.

Part II — the shape was the same on both papers

[A]

`count_digit(n, d)`	23
`while`	
`alternating_sum(n)`	23
`for`	
`menu()`	3
`main()`	20
`main()` called	1

[B]

`count_even_digits(n)`	23
`while`	
`sum_even_squares(n)`	23
`for`	
`menu()`	3
`main()`	20
`main()` called	1

Three rules both versions stated: the functions **return**, they never `print` or `input`; *all* reading and *all* validation happen in `main()`; the loop you were told to use is the loop you must use — a correct `for` where a `while` was required does not score the 23.

```python
# returns how many times d appears in n
def count_digit(n, d):
    if n == 0:
        if d == 0:
            return 1
        return 0
    count = 0
    # stops when there are no digits left
    while n > 0:
        if n % 10 == d:
            count = count + 1
        n = n // 10
    return count
```

The idea: `% 10` gives you the last digit, `// 10` removes it. Repeat until nothing is left.

1212, d=2 → digits 2, 1, 2, 1 → 2

The trap: `count_digit(0, 0)` **must return 1.**

A plain `while n > 0` never runs for n=0 and returns 0. That is why the special case is written *before* the loop. The examples on the paper told you: `count_digit(0, 0)` → 1.

This is the difference between *Excelente* and *Muy bien* on this function.

```python
# returns how many digits of n are even
def count_even_digits(n):
    if n == 0:
        return 1
    count = 0
    # stops when there are no digits left
    while n > 0:
        if (n % 10) % 2 == 0:
            count = count + 1
        n = n // 10
    return count
```

Same machine: `% 10` to look at the last digit, `// 10` to drop it.

1020 → digits 0, 2, 0, 1 → three even → 3

Same trap: `count_even_digits(0)` must return 1 — zero has one digit and it is even.

Also watch: `(n % 10) % 2 == 0` needs its parentheses read correctly; `n % 10 % 2` happens to work (left to right), but write the parentheses — you are not testing your luck, you are writing something a human reads.

[A] alternating_sum(n): $1 - 2 + 3 - 4 \cdots$

```python
def alternating_sum(n):
    total = 0
    # runs n times
    for i in range(1, n + 1):
        if i % 2 == 1:
            total = total + i
        else:
            total = total - i
    return total
```

(5) $\rightarrow 1 - 2 + 3 - 4 + 5 = 3$

[B] sum_even_squares(n): $2^2 + 4^2 + \cdots$

```python
def sum_even_squares(n):
    total = 0
    # runs n times
    for i in range(1, n + 1):
        if i % 2 == 0:
            total = total + i ** 2
    return total
```

(6) $\rightarrow 4 + 16 + 36 = 56$

The trap on both: `range(1, n)` drops the last term — it must be `range(1, n + 1)`, since the problem said "including n". The examples on the paper were there to catch exactly this.

Part II · menu() and main() — the 24 points most people left behind

```python
def main():
    keep_going = True
    while keep_going:
        menu()
        option = int(input("Option: "))
        if option == 1:
            n = int(input("Number n: "))
            d = int(input("Digit d (0-9): "))
            if n < 0 or d < 0 or d > 9:
                print("Invalid input: ...")
            else:
                print("Times:", count_digit(n,d))
        elif option == 2:
            ...                  # same shape
        elif option == 3:
            print("Goodbye.")
            keep_going = False
        else:                    # invalid -> also ends
            print("Invalid option.")
            keep_going = False
main()
```

The five things worth points:

1. the `while` that repeats the menu
2. `if`/`elif`/`else` on the option
3. **validation** of every input, with a clear message
4. the **call** to each function and a **labelled** output
5. `main()` on the last line — 1 free point

What cost points: no loop (the menu runs once); validation inside the function instead of `main`; the function called but never printed; the free point lost by not calling `main()`.

Where the points went

Part I — the five recurring errors

1. forgetting the line that is *outside* the `if` (`end`)
2. `**` before `%` and `//`
3. `not` applied to the whole line
4. crossed arguments in the function call
5. `range` with a negative step: counting one line too many

Part II — the five that cost most

1. `print` inside the function instead of `return`
2. the wrong loop (`for` where `while` was required)
3. `range(1, n)` — the last term missing
4. the zero case not handled
5. no validation, or validation in the wrong place

And −2 each: variable names like `x`, inputs with no prompt, output with no label.

Every one of these is a *reading* error or a *checklist* error. Neither needs more theory — both need the paper in front of you and two minutes at the end.

What to do with this result

Tonight, 30 minutes

1. Redo *your* version's Part I on paper, with a trace table per question.
2. Type the Part II program yourself, from scratch, and run the examples that are printed on the paper.
3. Compare with the solution on the site — *after*, not before.

The midterm is 20 % of the course. The final is 30 % and the situación problema 15 %: more than half of your grade is still ahead of you.

On the site now

Both papers, both full solutions with the *Why* boxes, and a Python key you can run in Thonny — exams page.

Asesorías

Book a slot in my calendar — the link is in the Canvas announcement. Bring the paper.

If you are behind on more than the exam

Come and talk to me this week — not in December. There is more room to fix it now than you think.

The exam is not the verdict.

It is the list of what to fix.

Next: **lists** — the first data structure.

Quick exam 5 next Tuesday · Situación problema deliverable 1 due Friday 6 November

What is still ahead of you

	weight	where it stands
Midterm	20 %	in your hand, graded
Quick exams	12 %	four sat, one left
Laboratories	10 %	running
Homework · in-class	8 %	running
Adaptive module · diagnostic	5 %	running
Situación problema	**15 %**	**not a single point spent**
Final exam	**30 %**	**Tuesday 1 December**

45 %

has not been won or lost yet.

Of those two, the project is the one you control most: four dates, a published rubric, no timer, and you may use the book, the site and each other.

The exam is 80 minutes. The project is four weeks.

The situación problema — what it actually is

You pick a real business and a real **dataset** — a table of data somebody actually collected. By the end of November you hand in a Python program that loads that table, computes statistics from it and draws two charts answering questions *you* wrote, plus a short report saying what the business should do about it.

There is no new theory in it:

- `menu()` + `main()` with a `while` — the lab
- `if`/`elif`/`else`, functions that **return** — week 2
- lists and plots — weeks 7 and 8
- data frames — week 9

Who hands in what

Deliverables 1, 2 and 3: **teams of two**, one grade for both of you. The **final delivery is individual** — your own program, your own report, two minutes explaining your own code out loud.

Two words, all month

Dataset: the table of real data you choose, as an Excel file.

Data frame: the same table once Python holds it in memory (`tablero` in the brief) — week 9.

Brief and rubric: **lrodriguez15.github.io**

The four deliveries — every date

	Due 23:59	What you hand in
1	**Fri 6 Nov** team of 2	An Excel table from a real, named source — a link, or a letter from the business (a family business counts). **30–50 rows**, up to 10 columns, about half text and half numbers, clean. Plus half a page: what the business is, what each column means. → `entrega1.xlsx`
2	**Fri 13 Nov** team of 2	Four to six *preguntas detonadoras* — **driving questions** a manager really asks (*which branch sells most? which product?*), each answerable **from your own columns** and each naming the column and the statistic or chart that answers it. Two or three per person. → `entrega2.docx`
3	**Fri 20 Nov** team of 2	A program that loads `entrega1.xlsx` into a data frame and prints it — clean run, no errors, a function plus `main()`. The printed table matches the Excel file. → `entrega3.py`
F	**Fri 27 Nov** **individual**	The whole program, the report, and a two-minute oral explanation of your own code. → `entregaF_matricula.py` + report

All four: **Canvas** → **Modules** → **Situación problema**, never email. **Every stage missed or late costs 5 points** off the project grade.

The final delivery — where its 100 points are

Program	**50**
`estadistica_descriptiva` — five or more statistics, tied to your questions	12
`grafica1` and `grafica2` — title, axis labels, legend	12
`menu()` + `main()`: the `while`, the `if/elif/else`, every `input()` in `main`	12
functions that **return**; a comment before each	8
runs from a clean start on your own file	6
Report	**30**
Oral explanation — two minutes, no notes	**20**

The report, four things

1. a cover, the business, and what each column means
2. your driving questions **answered** — your numbers, screenshots of your charts
3. a conclusion: what the analysis shows
4. a **strategic proposal** — what the business should do about it

Why the oral exists. Pick any function and say what it does and why it is written that way. **A program you cannot explain is not yours**: under 10 caps the program at 25.

November, on one page

Mon	Tue	Wed	Thu	Fri	Sat	Sun
2	3 · today papers back lists	4	5	6 SP 1: dataset	7	8 HW 6 · Lab 6
9	10 Quick exam 5 lists · in class	11	12	13 SP 2: questions	14	15 HW 7 · Lab 7
16	17	18	19	20 SP 3: data frame	21	22 HW 8 · Lab 8
23	24	25	26	27 SP final individual	28	29 Lab 9
30	1 Dec FINAL EXAM in class					

Every Friday and Sunday box is a **Canvas deadline at 23:59**, Querétaro time. Every Tuesday box happens **in this room**. Red is a date that cannot move.

If the midterm went badly — this week, in this order

If your mark starts with a 3, a 4 or a 5, this frame is for you. It is recoverable arithmetic — but only if this week looks different from the last one.

1. **Book an asesoría with me this week** — the booking link is in the Canvas announcement. One slot a week until the final. Bring the paper and what did not work.
2. **Use the site.** Both papers, both full solutions with the *Why* boxes, and **three practice midterms with solutions — lrodriguez15.github.io**, exams page.
3. **One program a day, on a timer.** 25 minutes, blank file, no help, then check.
4. **Never hand in an empty programming box.** In quick exams 1 and 2 about a quarter of this group left it blank — and that box is **50 of the 100 points**. A wrong attempt scores something; an empty box cannot.
5. **Find your partner and a dataset before Friday.**

Tonight, one thing:

open three datasets and keep the one whose columns you can explain.

Do that and Friday is nearly done — and the other 45 % starts tonight instead of in the last week of November.

Teams of two on the board before you leave · Deliverable 1 · Friday 6 November, 23:59

The rest of the course — four weeks, then the final

Wk	Tue / Fri	Topic	The new hard idea
7	**Nov 3 / 6**	**Lists**	one name, many values, **in order** — the last 20 minutes of today
8	Nov 10 / 13	**Plots**	a chart is an argument, not a decoration
9	Nov 17 / 20	**Import and extract data**	real data is dirty
10	Nov 24 / 27	**Arrays**	vectorised vs element-by-element
11	**Dec 1** / Dec 4	**FINAL EXAM** then the debrief	a Tuesday, not a Friday — 30 % of the course

Four topics left, and not one of them is harder than the `while` loop you already survived. Each is a new *kind* of thing to put in a variable — never a new way of thinking. **Quick exam 5: Tuesday 10 November**, on the previous week's topic, as always — which is **this** week's: lists.

Each of those weeks hands you the next piece of the project

Week	What it gives your program
7 lists	the column of numbers your statistics run on → `estadistica_descriptiva`
8 plots	`grafica1` and `grafica2` — title, axis labels, legend
9 data	`entrega1.xlsx` loaded into a data frame → `entrega3.py`
10 arrays	a whole column at once instead of one row at a time

Look at the four deadlines again and then at this table: **every stage is due the Friday of the week that teaches it.**

Nothing in the project is taught *after* you need it — and nothing in it is taught twice. Miss a Tuesday and you are not behind on theory, you are behind on your own program.

The only piece you need *before* any of this is the one piece no lecture can give you: **the dataset**. That is why it is due first.

One name, one value — and then the dataset arrives

```
venta1 = 12.5
venta2 = 9.0
venta3 = 14.2
# ... and 27 more lines

total = venta1 + venta2 + venta3
# ... + 27 more terms

print(total / 30)
```

Deliverable 1 asks for **30 to 50 rows**. Every variable you have written holds **one** value.

So count the damage:

- 30 names to invent, type and keep in order
- one more row → you edit the program
- you cannot `for` over 30 separate names
- `max`? `min`? by hand, 30 comparisons

Not a style problem — the end of the road: with one value per name you cannot write a program for a table you have not seen yet.

A list is one name for many values, in order

```
1 ventas = [12.5, 9.0, 14.2, 7.8]
2
3 print(ventas)
4 print(ventas[0])
5 print(ventas[3])
6 print(len(ventas))
```

prints `[12.5, 9.0, 14.2, 7.8]` `12.5` `7.8` `4`

Two words, for the rest of the course

List (*lista*): one name, many values, **in the order you put them**. Square brackets to build it, commas between the values.

Index (*índice*): the position of one value inside the list. Square brackets again, this time to *read*.

The index starts at 0.

value	12.5	9.0	14.2	7.8
index	**0**	**1**	**2**	**3**

`len(ventas)` is **how many**, not *where*.

The last index is `len` minus one

```
ventas = [12.5, 9.0, 14.2, 7.8]

print(len(ventas))                # 4
print(ventas[len(ventas)])        # IndexError
print(ventas[len(ventas) - 1])    # 7.8
print(ventas[-1])                 # 7.8 again
print(ventas[-2])                 # 14.2
```

`IndexError: list index out of range` — the program *stops* there. Nothing after that line runs.

Four values. Positions **0, 1, 2, 3**. There is no position 4.

- first: `ventas[0]`
- last: `ventas[len(ventas) - 1]`
- last, the short way: `ventas[-1]`
- never `ventas[len(ventas)]`

You have met this exact error already.

On the midterm it was `range(1, n)` dropping the last term. Same boundary, different hat: **the count is always one more than the last position.**

Two ways to walk a list — and when each is right

The value — when you only need *what*:

```
for v in ventas:
    print(v)
```

The position — when you need *where*:

```
for i in range(len(ventas)):
    print(i + 1, ventas[i])
```

Use the first for a total, a maximum, a count — anything where the position does not matter. It cannot go out of range: there is no index to get wrong.

Use the second when you need the row number, or when two lists run side by side: `sucursales[i]` sold `ventas[i]`.

`range(len(ventas))` gives `0, 1, 2, 3` — it stops *before* `len`, which is exactly the valid set of indices. **That is why it is written that way and never** `range(1, len(ventas))`.

append() — the list that grows inside the loop

```python
ventas = []           # empty, on purpose

for i in range(5):
    x = float(input("Sale: "))
    ventas.append(x)

print(ventas)
print(len(ventas), "values read")
```

You already know this shape. In week 3 it was `total = 0` *before* the loop and `total = total + x` *inside* it.

Same rule, new container: `ventas = []` before, `ventas.append(x)` inside.

- `[]` is a real list with nothing in it — `len` is 0
- `append` adds **at the end**: the order is the order you read
- put it *inside* the loop and you throw it away every turn

The accumulator you debugged in week 3, holding values instead of a sum.

Your first project function, on a list

```python
# one column of your dataset, by hand for now
ventas = [12.5, 9.0, 14.2, 7.8, 11.1]
# returns the mean of a list of numbers
def media(datos):
    total = 0
    for v in datos:
        total = total + v
    return total / len(datos)

def main():
    print("rows:", len(ventas))
    print("max :", max(ventas))
    print("min :", min(ventas))
    print("mean:", media(ventas))

main()
```

prints `rows: 5` `max : 14.2` `min : 7.8` `mean : 10.92`

Nothing here is new: a function that **returns**, a `for` with an accumulator, `main()` doing the printing.

`max`, `min` **and** `sum` **already work on a list** — write `media` by hand once anyway: the rubric asks you to *compute* the statistics.

In week 9 `ventas` stops being typed by hand — it becomes a **column of your Excel file**, and this grows into `estadistica_descriptiva(tablero)`: same loop, five statistics.

Before Friday, one thing:

put ten numbers from your own column into a list

and make it print max, min and mean.

Nine lines. If it runs, the first function of your project exists
— and Friday's lab is practice, not discovery.

Dataset **Friday 6 November, 23:59** · Quick exam 5 on lists **Tuesday 10 November**